
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 8

Caracteres e Strings em C

Exercícios de Autorrevisão (8.1–8.4)

Resolvidos passo a passo, abordagem incremental

Capítulos 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8

Construindo sobre os Capítulos Anteriores

No Capítulo 8, aprofundamos o trabalho com texto – ampliando os ponteiros do Cap. 7 para abordar caracteres e strings sistematicamente. Construindo sobre os Capítulos 1–7, incorporamos:

- **Biblioteca <ctype.h>** – funções de teste e conversão de caracteres: `isdigit`, `isalpha`, `isalnum`, `isxdigit`, `islower`, `isupper`, `isspace`, `iscntrl`, `isprint`, `ispunct`, `tolower`, `toupper`
- **Biblioteca <stdlib.h>** – conversão de strings para números: `atoi`, `atof`, `atol`, `strtod`, `strtol`
- **Biblioteca <string.h>** – manipulação de strings: `strcpy`, `strncpy`, `strcat`, `strncat`, `strcmp`, `strncmp`, `strlen`, `strchr`, `strrchr`, `strstr`, `strpbrk`, `strtok`, `strspn`, `strcspn`, `memcpy`, `memset`, `memcmp`, `memchr`
- **Biblioteca <stdio.h>** estendida – E/S de caracteres: `getchar`, `putchar`, `puts`, `gets`/`fgets`, `sprintf`, `scanf`

Boa prática de programação

As bibliotecas-padrão de C oferecem dezenas de funções otimizadas para caracteres e strings. **Sempre** use essas funções em vez de reimplementá-las – além de evitar bugs, você ganha décadas de otimização contínua dos compiladores.

Contents

1	Exercício 8.1 – Instruções para Tarefas Diversas	2
2	Exercício 8.2 – Duas formas de inicializar vogal	4
3	Exercício 8.3 – Análise de Instruções	4
4	Exercício 8.4 – Identificar e Corrigir Erros	7

1. Exercício 8.1 – Instruções para Tarefas Diversas

Resposta detalhada

Considere as variáveis:

```
1 int c, x, y, z;  
2 double d, e, f;  
3 char *ptr;  
4 char s1[ 100 ], s2[ 100 ];
```

a) Converter c em maiúscula:

```
1 c = toupper( c );
```

b) Verificar se c é dígito (operador condicional):

```
1 printf( "'%c'%sdigit\n", c,  
2         isdigit( c ) ? " e um " : " nao e um " );
```

c) Converter “1234567” em long:

```
1 printf( "%ld\n", atol( "1234567" ) );
```

d) Verificar se c é caractere de controle:

```
1 printf( "'%c'%scaractere de controle\n", c,  
2         iscntrl( c ) ? " e um " : " nao e um " );
```

e) Ler uma linha para s1 sem scanf:

```
1 fgets( s1, 100, stdin );
```

f) Imprimir s1 sem printf:

```
1 puts( s1 );
```

g) Localizar a última ocorrência de c em s1:

```
1 ptr = strrchr( s1, c );
```

h) Imprimir o valor de c sem printf:

```
1 putchar( c );
```

i) Converter “8.63582” em double:

```
1 printf( "%f\n", atof( "8.63582" ) );
```

j) Verificar se c é letra:

```
1 printf( "'%c'%sletra\n", c,  
2         isalpha( c ) ? " e uma " : " nao e uma " );
```

k) Ler caractere do teclado para c:

```
1 c = getchar();
```

l) Localizar primeira ocorrência de s2 em s1:

```
1 ptr = strstr( s1, s2 );
```

m) Verificar se c é imprimível:

```
1 printf( "'%c'%scaractere imprimivel\n", c,  
2         isprint( c ) ? " e um " : " nao e um " );
```

n) Ler 3 double de uma string:

```
1 sscanf( "1.27 10.3 9.432", "%lf%lf%lf", &d, &e, &f  
        );
```

Nota: %lf é usado para double no sscanf (não %f).

o) Copiar s2 para s1:

```
1 strcpy( s1, s2 );
```

p) Primeira ocorrência de qualquer caractere de s2 em s1:

```
1 ptr = strpbrk( s1, s2 );
```

q) Comparar s1 e s2:

```
1 printf( "strcmp( s1, s2 ) = %d\n", strcmp( s1, s2 )  
        );
```

r) Primeira ocorrência de c em s1:

```
1 ptr = strchr( s1, c );
```

s) Imprimir x, y, z em s1 com sprintf e largura 7:

```
1 sprintf( s1, "%7d%7d%7d", x, y, z );
```

t) Acrescentar 10 caracteres de s2 a s1:

```
1 strncat( s1, s2, 10 );
```

u) Comprimento de s1:

```
1 printf( "strlen(s1) = %u\n", strlen( s1 ) );
```

v) Converter “-21” em int:

```
1 printf( "%d\n", atoi( "-21" ) );
```

w) Primeiro token de s2 (separador: vírgula):

```
1 ptr = strtok( s2, "," );
```

Boa prática de programação

As funções de <string.h> e <ctype.h> formam o *kit básico* para qualquer programa que manipule texto. Memorizar pelo menos as mais comuns – `strcpy`, `strcmp`, `strlen`, `strcat`, `strchr`, `strstr`, `strtok` – é fundamental.

2. Exercício 8.2 – Duas formas de inicializar vogal

Resposta detalhada

Método 1 – Usando literal de string:

```
1 char vogal [] = "AEIOU";
```

O compilador automaticamente:

- Conta os caracteres: 5 letras.
- Adiciona o terminador '\0' ao final.
- Dimensiona o array com 6 elementos.

Método 2 – Usando lista de caracteres explícita:

```
1 char vogal [] = { 'A', 'E', 'I', 'O', 'U', '\0' };
```

Aqui devemos colocar **explicitamente** o '\0' no final – sem ele, a string não seria válida e funções como `strlen` ou `printf` com `%s` se comportariam de modo imprevisível.

Erro comum de programação

Esquecer o '\0' ao inicializar manualmente uma string é um erro grave. Funções de `<string.h>` continuam lendo memória após o fim do array até encontrar (por acidente) um byte zero – causando saídas estranhas, vazamento de informação ou falha de segmentação.

3. Exercício 8.3 – Análise de Instruções

Considere: `char s1[50] = "jack", s2[50] = " jill", s3[50], *sptr;`

Item (a)

Resposta detalhada

```
1 printf( "%c%s", toupper( s1[ 0 ] ), &s1[ 1 ] );
```

Análise:

- `s1[0]` é 'j'; `toupper('j')` retorna 'J'.
- `&s1[1]` é o endereço do segundo caractere de `s1`, ou seja, "ack" (porque `&s1[1]` aponta para o 'a').

Saída: Jack

Item (b)

Resposta detalhada

```
1 printf( "%s", strcpy( s3, s2 ) );
```

Análise: strcpy(s3, s2) copia " jill" para s3 e retorna o ponteiro para o destino s3.

Saída: jill (note o espaço inicial)

Item (c)

Resposta detalhada

```
1 printf( "%s", strcat( strcat( strcpy( s3, s1 ), " and  
" ), s2 ) );
```

Análise passo a passo:

a) strcpy(s3, s1) → s3 = "jack", retorna s3.

b) strcat(s3, " and ") → s3 = "jack and ", retorna s3.

c) strcat(s3, s2) → s3 = "jack and jill" (note o duplo espaço, pois s2 começa com espaço).

Saída: jack and jill

Resposta do livro: “jack e jill” – assume tradução de “and” → “e” e ignora o espaço duplo. O resultado real do código é com “and” literal.

Item (d)

Resposta detalhada

```
1 printf( "%u", strlen( s1 ) + strlen( s2 ) );
```

Análise:

- strlen("jack") = 4
- strlen(" jill") = 5 (5 caracteres incluindo o espaço inicial)
- Soma: 4 + 5 = 9

Resposta do livro: 8. Há uma pequena discrepância: a resposta do livro considera strlen(" jill") = 4 (sem contar o espaço), mas o espaço faz parte da string e deveria ser contado. **Saída real:** 9.

Item (e)

Resposta detalhada

```
1 printf( "%u", strlen( s3 ) );
```

Após o item (c), s3 = "jack and jill" (14 caracteres incluindo os 2 espaços entre “and” e “jill”).

Saída: 14 (livro diz 13; novamente uma pequena discrepância sobre

contagem de espaços).

4. Exercício 8.4 – Identificar e Corrigir Erros

Item (a)

Resposta detalhada

Código com erro:

```
1 char s[ 10 ];
2 strncpy( s, "ola", 5 );
3 printf( "%s\n", s );
```

Erro: `strncpy` não adiciona automaticamente o terminador `'\0'` se o terceiro argumento for menor ou igual ao comprimento da string copiada. Quando o terceiro argumento é maior que a string de origem, `strncpy` preenche com zeros, então neste caso “ola” + 2 zeros extras OK. Porém, se a string fosse maior que 5, faltaria o terminador.

Cuidado clássico: se a string for “hello” (5 letras) e copiamos exatamente 5, não há `'\0'`.

Correção (caso geral): sempre adicionar manualmente o terminador após `strncpy`:

```
1 strncpy( s, "ola", 5 );
2 s[ 5 ] = '\0'; /* garantir terminador */
```

Item (b)

Resposta detalhada

Código com erro:

```
1 printf( "%s", 'a' );
```

Erro: `'a'` é uma *constante de caractere* (um `int` com valor 97 no ASCII), não uma string. Usar `%s` espera um ponteiro para `char` – usar 97 como ponteiro causa falha de segmentação.

Correções possíveis:

```
1 printf( "%c", 'a' ); /* %c para caractere individual
   */
2 printf( "%s", "a" ); /* "a" e uma string (com
   terminador) */
```

Item (c)

Resposta detalhada

Código com erro:

```
1 char s[ 12 ];
2 strcpy( s, "Bem-vindo" );
```

Erro: A string “Bem-vindo” tem 9 caracteres + `'\0'` = 10 bytes. Cabe em `s[12]`? Sim, mas se levarmos em conta que a string em português poderia ter caractere acentuado (codificação multi-byte em UTF-8), a contagem pode mudar. Em ASCII puro, está OK – mas tradicionalmente o livro usa exemplos onde o array é *pequeno demais*.

Considerando a observação do livro: a tradução para inglês seria “Welcome” (7 + 1 = 8 bytes – cabe). Se a string fosse “Welcome to the show” (20 chars + `'\0'`), não caberia em 12.

Correção: declarar o array com tamanho suficiente:

```
1 char s[ 50 ];    /* tamanho confortavel */
2 strcpy( s, "Bem-vindo" );
```

Erro comum de programação

Buffer overflow: copiar uma string maior que o destino é uma das vulnerabilidades de segurança mais exploradas em programas C. As versões `strncpy` e `strncat` ajudam, mas exigem cuidado com o terminador.

Item (d)

Resposta detalhada

Código com erro:

```
1 if ( strcmp( string1, string2 ) ) {
2     printf( "As strings sao iguais\n" );
3 }
```

Erro: A função `strcmp` retorna 0 quando as strings são iguais. No `if`, 0 é falso – então a mensagem *nunca* é impressa quando as strings são iguais. A lógica está invertida!

Correção: comparar explicitamente com 0:

```
1 if ( strcmp( string1, string2 ) == 0 ) {
2     printf( "As strings sao iguais\n" );
3 }
```

Boa prática de programação

Sempre compare o retorno de `strcmp` com `== 0` (para igualdade), `< 0` (`s1` vem antes de `s2`) ou `> 0` (`s1` vem depois de `s2`). Nunca trate `strcmp` como um booleano direto – é uma armadilha clássica.

Gabarito Rápido – Capítulo 8: Autorrevisão

Respostas Resumidas

8.1 – 23 funções/operações principais com strings e caracteres:
toupper, isdigit, atol, iscntrl, fgets, puts, strrchr, putchar, atof, isalpha, getchar, strstr, isprint, sscanf, strcpy, strpbrk, strcmp, strchr, sprintf, strncat, strlen, atoi, strtok.

8.2: Duas formas de inicializar:

- `char vogal[] = "AEIOU";` – compilador adiciona `'\0'`.
- `char vogal[] = { 'A', 'E', 'I', 'O', 'U', '\0' };` – terminador explícito.

8.3 – Saídas das instruções:

- Jack
- jill
- jack and jill
- 9 (sem ignorar espaços)
- 14

8.4 – Erros corrigidos:

- `strncpy` pode não adicionar `'\0'`; adicione manualmente.
- `'a'` não é string; use `%c` ou `"a"`.
- Array pode ser pequeno demais; declare com tamanho seguro.
- Inverteu lógica: `strcmp` retorna 0 quando iguais (`compare == 0`).